

DL_practica_pytorch_a_DQN

March 23, 2026

1 Práctica: De PyTorch Básico a DQN para Videojuegos

1.1 Objetivos

En esta sesión práctica de **2 horas** trabajarás progresivamente desde los fundamentos de PyTorch hasta entrenar una red neuronal que aprenda a jugar un videojuego.

1.1.1 Estructura

Parte	Contenido	Tiempo
1	PyTorch Básico: Tensores y Operaciones	20 min
2	Autograd y Redes Neuronales	25 min
3	Entrenamiento: Clasificador Iris	25 min
4	Juegos y Problemáticas de RL	20 min
5	DQN: Tu agente jugando	30 min

1.1.2 Instrucciones

- Los ejercicios tienen **código para completar** marcado con **# TODO**:
- Ejecuta las celdas de **verificación** para comprobar tus respuestas
- Si te atascas, hay **pistas** disponibles

```
[53]: # Imports necesarios
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from collections import deque
import random

# Configuración
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"PyTorch: {torch.__version__}")
print(f"Dispositivo: {DEVICE}")
```

```
# Semilla para reproducibilidad
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)
```

PyTorch: 2.10.0+cpu

Dispositivo: cpu

2 PARTE 1: PyTorch Básico - Tensores (20 min)

Los **tensores** son la estructura fundamental de PyTorch. Son como arrays de NumPy pero con soporte para GPU y diferenciación automática.

2.1 Ejercicio 1.1: Creación de Tensores

Crea los siguientes tensores:

```
[54]: # EJERCICIO 1.1: Crea los tensores indicados

# a) Un tensor de ceros de forma (3, 4)
tensor_ceros = torch.zeros(3, 4) # TODO: usa torch.zeros()

# b) Un tensor de unos de forma (2, 3)
tensor_unos = torch.ones(2, 3) # TODO: usa torch.ones()

# c) Un tensor con valores aleatorios normales de forma (5, 5)
tensor_random = torch.randn(5, 5) # TODO: usa torch.randn()

# d) Un tensor a partir de una lista de Python: [1, 2, 3, 4, 5]
tensor_lista = torch.tensor([1, 2, 3, 4, 5]) # TODO: usa torch.tensor()

[55]: # VERIFICACIÓN 1.1
assert tensor_ceros is not None and tensor_ceros.shape == (3, 4), "tensor_ceros_
↳debe ser (3,4)"
assert tensor_ceros.sum() == 0, "tensor_ceros debe contener solo ceros"
assert tensor_unos is not None and tensor_unos.shape == (2, 3), "tensor_unos_
↳debe ser (2,3)"
assert tensor_unos.sum() == 6, "tensor_unos debe contener solo unos"
assert tensor_random is not None and tensor_random.shape == (5, 5),
↳"tensor_random debe ser (5,5)"
assert tensor_lista is not None and list(tensor_lista) == [1, 2, 3, 4, 5],
↳"tensor_lista incorrecto"
print("Ejercicio 1.1 CORRECTO")
```

Ejercicio 1.1 CORRECTO

2.2 Ejercicio 1.2: Operaciones con Tensores

Realiza las operaciones indicadas:

```
[56]: # EJERCICIO 1.2: Operaciones básicas

a = torch.tensor([1.0, 2.0, 3.0, 4.0])
b = torch.tensor([2.0, 2.0, 2.0, 2.0])

# a) Suma elemento a elemento de a + b
suma = a + b # TODO

# b) Multiplicación elemento a elemento de a * b
mult = a * b # TODO

# c) Media de todos los elementos de 'a'
media = a.mean() # TODO: usa .mean()

# d) Producto punto (dot product) de a y b
dot = torch.dot(a, b) # TODO: usa torch.dot() o @

[57]: # VERIFICACIÓN 1.2
assert suma is not None and list(suma) == [3.0, 4.0, 5.0, 6.0], "suma_
↳incorrecta"
assert mult is not None and list(mult) == [2.0, 4.0, 6.0, 8.0], "mult_
↳incorrecta"
assert media is not None and media.item() == 2.5, "media incorrecta"
assert dot is not None and dot.item() == 20.0, "dot product incorrecto"
print("Ejercicio 1.2 CORRECTO")
```

Ejercicio 1.2 CORRECTO

2.3 Ejercicio 1.3: Reshape y Slicing

Manipula la forma de los tensores:

```
[58]: # EJERCICIO 1.3: Reshape y slicing

# Tensor original de forma (2, 6)
original = torch.arange(12).reshape(2, 6).float()
print(f"Original (2, 6):\n{original}\n")

# a) Reorganiza a forma (3, 4)
reshape_3x4 = original.reshape(3, 4) # TODO: usa .reshape() o .view()

# b) Reorganiza a forma (12,) - vector plano
flatten = original.reshape(-1) # TODO: usa .reshape(-1) o .flatten()

# c) Extrae la primera fila del tensor original
```

```
primera_fila = original[0, :] # TODO: slicing [?, ?]

# d) Extrae las columnas 2, 3, 4 (índices 2:5) del tensor original
columnas_234 = original[:, 2:5] # TODO: slicing[:, ?:]
```

```
Original (2, 6):
tensor([[ 0.,  1.,  2.,  3.,  4.,  5.],
        [ 6.,  7.,  8.,  9., 10., 11.]])
```

```
[59]: # VERIFICACIÓN 1.3
assert reshape_3x4 is not None and reshape_3x4.shape == (3, 4), "reshape_3x4_
↳debe ser (3,4)"
assert flatten is not None and flatten.shape == (12,), "flatten debe ser (12,)"
assert primera_fila is not None and list(primera_fila) == [0, 1, 2, 3, 4, 5],_
↳"primera_fila incorrecta"
assert columnas_234 is not None and columnas_234.shape == (2, 3), "columnas_234_
↳debe ser (2,3)"
print("Ejercicio 1.3 CORRECTO")
```

Ejercicio 1.3 CORRECTO

2.4 Ejercicio 1.4: Dispositivo (CPU/GPU)

Mueve tensores entre CPU y GPU:

```
[60]: # EJERCICIO 1.4: Mover tensores al dispositivo

# Crear tensor en CPU
tensor_cpu = torch.randn(100, 100)
print(f"Tensor original en: {tensor_cpu.device}")

# a) Mueve el tensor al dispositivo disponible (DEVICE)
tensor_device = tensor_cpu.to(DEVICE) # TODO: usa .to(DEVICE)

# b) Crea un tensor directamente en el dispositivo correcto
tensor_directo = torch.randn(3, 4, device=DEVICE) # TODO: usa torch.randn(...,_
↳device=DEVICE)
```

Tensor original en: cpu

```
[61]: # VERIFICACIÓN 1.4
assert tensor_device is not None and tensor_device.device.type == DEVICE.type,_
↳"tensor_device debe estar en DEVICE"
assert tensor_directo is not None and tensor_directo.device.type == DEVICE._
↳type, "tensor_directo debe estar en DEVICE"
print(f"Ejercicio 1.4 CORRECTO - Tensores en {DEVICE}")
```

Ejercicio 1.4 CORRECTO - Tensores en cpu

3 PARTE 2: Autograd y Redes Neuronales (25 min)

Autograd es el motor de diferenciación automática de PyTorch. Calcula gradientes automáticamente para entrenar redes neuronales.

3.1 Ejercicio 2.1: Gradientes Básicos

Calcula gradientes manualmente:

```
[62]: # EJERCICIO 2.1: Calcular gradientes

# Función:  $y = x^2 + 3x + 1$ 
# Derivada:  $dy/dx = 2x + 3$ 
# En  $x=2$ :  $dy/dx = 2(2) + 3 = 7$ 

# a) Crea un tensor  $x=2.0$  con requires_grad=True
x = torch.tensor(2.0, requires_grad=True) # TODO: torch.tensor(2.0, requires_grad=True)

# b) Calcula  $y = x^2 + 3x + 1$ 
y = x*x + 3*x + 1 # TODO

# c) Calcula el gradiente llamando a backward()
# TODO: llama a y.backward()
y.backward()

# d) El gradiente está en x.grad
gradiente = x.grad # TODO: asigna x.grad

[63]: # VERIFICACIÓN 2.1
assert x is not None and x.requires_grad, "x debe tener requires_grad=True"
assert y is not None and y.item() == 11.0, "y = 4 + 6 + 1 = 11"
assert gradiente is not None and gradiente.item() == 7.0, "dy/dx en x=2 debe ser 7"
print("Ejercicio 2.1 CORRECTO")
print(f"y = {y.item()}, dy/dx = {gradiente.item()}")
```

Ejercicio 2.1 CORRECTO

y = 11.0, dy/dx = 7.0

3.2 Ejercicio 2.2: Tu Primera Red Neuronal

Crea una red neuronal simple usando `nn.Sequential`:

Input (4) -> Linear(4, 16) -> ReLU -> Linear(16, 8) -> ReLU -> Linear(8, 3) -> Output (3)

```
[64]: # EJERCICIO 2.2: Crear red con nn.Sequential
```

```
# Crea la red siguiendo la arquitectura:  
# Input(4) -> Linear(16) -> ReLU -> Linear(8) -> ReLU -> Linear(3)  
  
red_simple = nn.Sequential(  
    nn.Linear(4, 16),  
    nn.ReLU(),  
    nn.Linear(16, 8),  
    nn.ReLU(),  
    nn.Linear(8, 3)  
)
```

```
[65]: # VERIFICACIÓN 2.2
```

```
test_input = torch.randn(1, 4) # Batch de 1, 4 features  
test_output = red_simple(test_input)  
assert test_output.shape == (1, 3), f"La salida debe ser (1, 3), pero es_{  
    ↪test_output.shape}"  
n_params = sum(p.numel() for p in red_simple.parameters())  
assert 200 < n_params < 300, f"Número de parámetros inesperado: {n_params}"  
print(f"Ejercicio 2.2 CORRECTO")  
print(f"Parámetros: {n_params}")  
print(f"Test: {test_input.shape} -> {test_output.shape}")
```

Ejercicio 2.2 CORRECTO

Parámetros: 243

Test: torch.Size([1, 4]) -> torch.Size([1, 3])

3.3 Ejercicio 2.3: Red como Clase (nn.Module)

Crea la misma red pero como clase que hereda de nn.Module:

```
[66]: # EJERCICIO 2.3: Crear red como clase
```

```
class MiRed(nn.Module):  
    def __init__(self, input_size, hidden1, hidden2, output_size):  
        super().__init__() # Siempre llamar al padre  
  
        # TODO: Define las capas como atributos  
        # self.fc1 = nn.Linear(...)  
        # self.fc2 = ...  
        # self.fc3 = ...  
        self.fc1 = nn.Linear(input_size, hidden1)  
        self.fc2 = nn.Linear(hidden1, hidden2)  
        self.fc3 = nn.Linear(hidden2, output_size)  
  
    def forward(self, x):  
        # TODO: Define el flujo de datos
```

```

        # x = F.relu(self.fc1(x))
        # ...
        # return x
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

# Instanciar la red
mi_red = MiRed(input_size=4, hidden1=16, hidden2=8, output_size=3)

```

```

[67]: # VERIFICACIÓN 2.3
test_input = torch.randn(5, 4) # Batch de 5
test_output = mi_red(test_input)
assert test_output.shape == (5, 3), f"La salida debe ser (5, 3), pero es {test_output.shape}"
assert hasattr(mi_red, 'fc1') or hasattr(mi_red, 'capa1'), "Debes definir las capas como atributos"
print(f"Ejercicio 2.3 CORRECTO")
print(mi_red)

```

Ejercicio 2.3 CORRECTO

```

MiRed(
  (fc1): Linear(in_features=4, out_features=16, bias=True)
  (fc2): Linear(in_features=16, out_features=8, bias=True)
  (fc3): Linear(in_features=8, out_features=3, bias=True)
)

```

3.4 Ejercicio 2.4: Forward y Backward Pass

Realiza un paso completo de forward y backward:

```

[68]: # EJERCICIO 2.4: Forward y Backward completo

# Red y datos
red = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3)
)

X = torch.randn(32, 4) # 32 muestras, 4 features
y_true = torch.randint(0, 3, (32,)) # 32 etiquetas (0, 1, o 2)

# a) Forward pass: obtén las predicciones
predicciones = red(X) # TODO: red(X)

```

```

# b) Calcula el loss con CrossEntropyLoss
criterio = nn.CrossEntropyLoss()
loss = criterio(predicciones, y_true) # TODO: criterio(predicciones, y_true)

# c) Backward pass: calcula gradientes
# TODO: loss.backward()
loss.backward()

# d) Verifica que los gradientes se calcularon
primer_peso = list(red.parameters())[0]
tiene_gradientes = primer_peso.grad is not None # TODO: primer_peso.grad is
↳ not None

```

```

[69]: # VERIFICACIÓN 2.4
assert predicciones is not None and predicciones.shape == (32, 3),
↳ "predicciones incorrectas"
assert loss is not None and loss.item() > 0, "loss debe ser positivo"
assert tiene_gradientes == True, "Los gradientes no se calcularon. ¿Llamaste a
↳ backward()?"
print(f"Ejercicio 2.4 CORRECTO")
print(f"Loss: {loss.item():.4f}")
print(f"Gradiente del primer peso: shape {primer_peso.grad.shape}")

```

Ejercicio 2.4 CORRECTO

Loss: 1.1242

Gradiente del primer peso: shape torch.Size([8, 4])

4 PARTE 3: Entrenamiento Completo - Clasificador Iris (25 min)

Ahora entrenaremos una red neuronal completa para clasificar el famoso **Iris Dataset**.

```

[70]: # Cargar el Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Cargar datos
iris = load_iris()
X, y = iris.data, iris.target

print(f"Iris Dataset:")
print(f"  Muestras: {X.shape[0]}")
print(f"  Features: {X.shape[1]} ({iris.feature_names})")
print(f"  Clases: {len(iris.target_names)} ({list(iris.target_names)})")

```


Iris Dataset:

Muestras: 150
Features: 4 (['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)'])
Clases: 3 ([np.str_('setosa'), np.str_('versicolor'), np.str_('virginica')])

4.1 Ejercicio 3.1: Preparar los Datos

```
[71]: # EJERCICIO 3.1: Preparar datos para PyTorch

# a) Dividir en train/test (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# b) Normalizar los datos (muy importante para redes neuronales)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# c) Convertir a tensores de PyTorch
X_train_t = torch.FloatTensor(X_train) # TODO: torch.FloatTensor(X_train)
y_train_t = torch.LongTensor(y_train) # TODO: torch.LongTensor(y_train) #
    ↪ Long para clasificación
X_test_t = torch.FloatTensor(X_test) # TODO
y_test_t = torch.LongTensor(y_test) # TODO

[72]: # VERIFICACIÓN 3.1
assert X_train_t is not None and X_train_t.shape == (120, 4), "X_train_t
    ↪ incorrecto"
assert y_train_t is not None and y_train_t.dtype == torch.long, "y_train_t debe
    ↪ ser LongTensor"
assert X_test_t is not None and X_test_t.shape == (30, 4), "X_test_t incorrecto"
print("Ejercicio 3.1 CORRECTO")
print(f"Train: {X_train_t.shape}, Test: {X_test_t.shape}")
```

Ejercicio 3.1 CORRECTO

Train: torch.Size([120, 4]), Test: torch.Size([30, 4])

4.2 Ejercicio 3.2: Crear la Red para Iris

```
[73]: # EJERCICIO 3.2: Crear red para clasificar Iris

class IrisClassifier(nn.Module):
    """
    Red para clasificar flores Iris.

    Arquitectura sugerida:
```

```

- Input: 4 features
- Hidden 1: 32 neuronas + ReLU
- Hidden 2: 16 neuronas + ReLU
- Output: 3 clases
"""

def __init__(self):
    super().__init__()
    # TODO: Define las capas
    # self.fc1 = nn.Linear(4, 32)
    # ...
    self.fc1 = nn.Linear(4, 32)
    self.fc2 = nn.Linear(32, 16)
    self.fc3 = nn.Linear(16, 3)
    self.relu = nn.ReLU()

def forward(self, x):
    # TODO: Define el forward pass
    # Recuerda: NO aplicar softmax al final
    # CrossEntropyLoss lo hace internamente
    x = self.fc1(x)
    x = self.relu(x)
    x = self.fc2(x)
    x = self.relu(x)
    x = self.fc3(x)

    return x

# Crear modelo
modelo_iris = IrisClassifier()

```

```

[74]: # VERIFICACIÓN 3.2
test_out = modelo_iris(X_train_t[:5])
assert test_out.shape == (5, 3), f"Salida debe ser (5, 3), es {test_out.shape}"
print("Ejercicio 3.2 CORRECTO")
print(modelo_iris)

```

Ejercicio 3.2 CORRECTO

```

IrisClassifier(
  (fc1): Linear(in_features=4, out_features=32, bias=True)
  (fc2): Linear(in_features=32, out_features=16, bias=True)
  (fc3): Linear(in_features=16, out_features=3, bias=True)
  (relu): ReLU()
)

```

4.3 Ejercicio 3.3: Ciclo de Entrenamiento

```
[75]: # EJERCICIO 3.3: Implementar el ciclo de entrenamiento

# Configuración
modelo = IrisClassifier()
criterio = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo.parameters(), lr=0.01)
epochs = 100

# Historial para graficar
historial_loss = []
historial_acc = []

# Ciclo de entrenamiento
for epoch in range(epochs):
    modelo.train() # Modo entrenamiento

    # TODO: Completa el ciclo de entrenamiento

    # 1. Forward pass
    predicciones = modelo(X_train_t) # TODO

    # 2. Calcular loss
    loss = criterio(predicciones, y_train_t) # TODO

    # 3. Backward pass
    # TODO: optimizador.zero_grad()
    # TODO: loss.backward()
    # TODO: optimizador.step()
    optimizador.zero_grad()
    loss.backward()
    optimizador.step()

    # 4. Calcular accuracy
    with torch.no_grad():
        _, predicted = predicciones.max(1)
        acc = (predicted == y_train_t).float().mean().item() * 100

    historial_loss.append(loss.item())
    historial_acc.append(acc)

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1:3d} | Loss: {loss.item():.4f} | Acc: {acc:.1f}%")
```

```
Epoch 20 | Loss: 0.3644 | Acc: 85.0%
Epoch 40 | Loss: 0.0812 | Acc: 96.7%
Epoch 60 | Loss: 0.0482 | Acc: 98.3%
```

Epoch 80 | Loss: 0.0403 | Acc: 98.3%
Epoch 100 | Loss: 0.0356 | Acc: 98.3%

```
[76]: # VERIFICACIÓN 3.3
assert len(historial_loss) == 100, "Debe haber 100 epochs"
assert historial_loss[-1] < historial_loss[0], "El loss debe disminuir"
assert historial_acc[-1] > 80, "El accuracy debe ser > 80% al final"
print("Ejercicio 3.3 CORRECTO")
print(f"Loss final: {historial_loss[-1]:.4f}")
print(f"Accuracy final: {historial_acc[-1]:.1f}%")
```

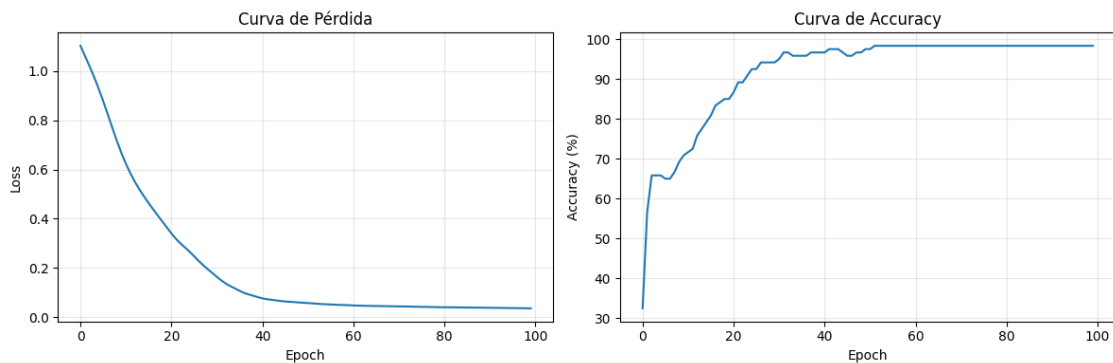
Ejercicio 3.3 CORRECTO
Loss final: 0.0356
Accuracy final: 98.3%

```
[77]: # Visualizar entrenamiento
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(historial_loss)
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Curva de Pérdida')
axes[0].grid(True, alpha=0.3)

axes[1].plot(historial_acc)
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy (%)')
axes[1].set_title('Curva de Accuracy')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



4.4 Ejercicio 3.4: Evaluación en Test

```
[78]: # EJERCICIO 3.4: Evaluar en el conjunto de test

modelo.eval() # Modo evaluación

with torch.no_grad(): # No calcular gradientes
    # TODO: Obtener predicciones en test
    predicciones_test = modelo(X_test_t) # TODO: modelo(X_test_t)

    # TODO: Calcular accuracy en test
    _, predicted_test = predicciones_test.max(1)
    test_acc = (predicted_test == y_test_t).float().mean().item() * 100 # TODO:
    ↪ calcular accuracy

[79]: # VERIFICACIÓN 3.4
assert test_acc is not None and test_acc > 80, "Accuracy en test debe ser > 80%"
print(f"Ejercicio 3.4 CORRECTO")
print(f"\n" + "="*50)
print(f"RESULTADO FINAL: {test_acc:.1f}% accuracy en test")
print(f"="*50)
```

Ejercicio 3.4 CORRECTO

```
=====
RESULTADO FINAL: 100.0% accuracy en test
=====
```

5 PARTE 4: Juegos y Problemáticas de RL (20 min)

Ahora pasamos de aprendizaje supervisado a **Reinforcement Learning**.

En RL, el agente aprende por **prueba y error**, recibiendo recompensas del entorno.

5.1 4.1 Diferencias Clave: Supervisado vs RL

Aspecto	Supervisado (Iris)	Reinforcement Learning
Datos	Dataset fijo con etiquetas	Generados por interacción
Feedback	Etiqueta correcta inmediata	Recompensa (puede ser delayed)
Objetivo	Minimizar error vs etiquetas	Maximizar recompensa total
Problema	i.i.d. samples	Datos correlacionados temporalmente

5.1.1 Problemáticas específicas de RL:

1. **Exploración vs Explotación:** ¿Probar cosas nuevas o usar lo que ya funciona?

2. **Recompensas sparse:** A veces solo hay recompensa al final del juego
3. **Correlación temporal:** Experiencias consecutivas son muy similares
4. **Moving target:** El objetivo cambia mientras aprendemos

5.2 4.2 Un Juego Sencillo: GridWorld

Crearemos un juego simple donde un agente debe llegar a la meta:

```
+---+---+---+---+
| A |   |   | G |
+---+---+---+---+
```

A = Agente (posición inicial)

G = Goal (meta, recompensa +10)

Acciones: 0=izquierda, 1=derecha

```
[80]: class SimpleGridWorld:
    """
    Juego GridWorld 1D simplificado.

    El agente empieza en posición 0 y debe llegar a posición 3.
    """

    def __init__(self):
        self.size = 4
        self.goal = 3
        self.position = 0
        self.max_steps = 10
        self.steps = 0

    def reset(self):
        """Reinicia el juego."""
        self.position = 0
        self.steps = 0
        return self._get_state()

    def _get_state(self):
        """Estado: one-hot de la posición."""
        state = np.zeros(self.size)
        state[self.position] = 1
        return state

    def step(self, action):
        """
        Ejecuta acción.

        action: 0 = izquierda, 1 = derecha
```

```

Returns: (next_state, reward, done)
"""
self.steps += 1

# Mover
if action == 0: # Izquierda
    self.position = max(0, self.position - 1)
else: # Derecha
    self.position = min(self.size - 1, self.position + 1)

# Recompensa y terminación
if self.position == self.goal:
    return self._get_state(), 10.0, True # ¡Victoria!
elif self.steps >= self.max_steps:
    return self._get_state(), -1.0, True # Timeout
else:
    return self._get_state(), -0.1, False # Penalización por paso

def render(self):
    """Visualiza el estado actual."""
    grid = ['_'] * self.size
    grid[self.goal] = 'G'
    grid[self.position] = 'A' if self.position != self.goal else '*'
    print(f"[{' '.join(grid)}] Pos: {self.position}")

# Probar el juego
print("=" * 50)
print("GridWorld Demo")
print("=" * 50)

env = SimpleGridWorld()
state = env.reset()
print("\nEstado inicial:")
env.render()

print("\nMoviendo a la derecha 3 veces:")
for _ in range(3):
    state, reward, done = env.step(1) # Derecha
    env.render()
    print(f"Reward: {reward}, Done: {done}")

```

```

=====
GridWorld Demo
=====

```

```

Estado inicial:
[A _ _ G] Pos: 0

```

Moviendo a la derecha 3 veces:

```
[_ A _ G] Pos: 1
  Reward: -0.1, Done: False
[_ _ A G] Pos: 2
  Reward: -0.1, Done: False
[_ _ _ *] Pos: 3
  Reward: 10.0, Done: True
```

5.3 Ejercicio 4.1: Agente Aleatorio

Primero, veamos cómo le va a un agente que actúa **aleatoriamente**:

```
[81]: # EJERCICIO 4.1: Ejecutar agente aleatorio

def agente_aleatorio(n_episodios=100):
    """Ejecuta un agente que toma acciones aleatorias."""
    env = SimpleGridWorld()
    recompensas_totales = []
    victorias = 0

    for episodio in range(n_episodios):
        state = env.reset()
        total_reward = 0
        done = False

        while not done:
            # TODO: Seleccionar acción aleatoria (0 o 1)
            action = random.randint(0, 1) # TODO: random.randint(0, 1)

            # TODO: Ejecutar acción
            state, reward, done = env.step(action) # TODO: env.step(action)

            total_reward += reward

        recompensas_totales.append(total_reward)
        if total_reward > 0:
            victorias += 1

    return recompensas_totales, victorias

# Ejecutar
rewards_random, wins_random = agente_aleatorio(100)
print(f"Agente Aleatorio (100 episodios):")
print(f"  Victorias: {wins_random}/100")
print(f"  Recompensa promedio: {np.mean(rewards_random):.2f}")
```


Agente Aleatorio (100 episodios):
Victorias: 61/100
Recompensa promedio: 5.07

5.4 4.3 Problema: Exploración vs Explotación

El dilema: - Si siempre exploras (aleatorio): nunca aprovechas lo aprendido - Si siempre explotas (greedy): puedes quedarte en un óptimo local

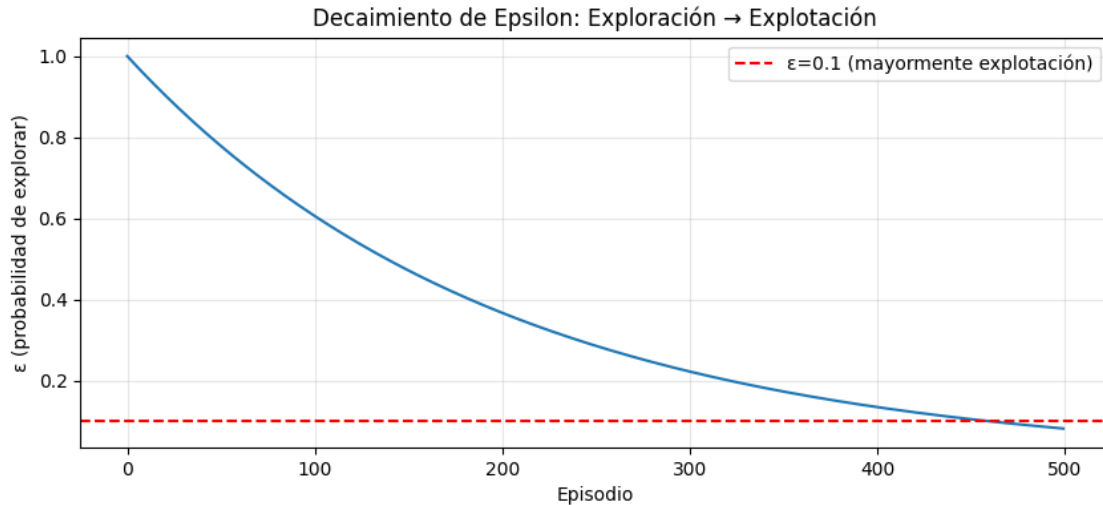
Solución: -**greedy** - Con probabilidad ϵ : acción aleatoria (explorar) - Con probabilidad $1-\epsilon$: mejor acción según el modelo (explotar) - ϵ decrece con el tiempo: explorar al inicio, explotar al final

```
[82]: # Visualizar decaimiento de epsilon
epsilon = []
epsilon = 1.0
epsilon_min = 0.01
epsilon_decay = 0.995

for _ in range(500):
    epsilon.append(epsilon)
    epsilon = max(epsilon_min, epsilon * epsilon_decay)

plt.figure(figsize=(10, 4))
plt.plot(epsilon)
plt.axhline(y=0.1, color='r', linestyle='--', label='=0.1 (mayormente_
    ↪explotación)')
plt.xlabel('Episodio')
plt.ylabel(' (probabilidad de explorar)')
plt.title('Decaimiento de Epsilon: Exploración → Explotación')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("Al inicio ( 1): El agente explora casi siempre")
print("Al final ( 0.01): El agente explota casi siempre")
```



Al inicio (1): El agente explora casi siempre
 Al final (0.01): El agente explota casi siempre

5.5 4.4 Problema: Experience Replay

El problema: Las experiencias consecutivas están muy correlacionadas.

Si el agente va: derecha, derecha, derecha → aprende solo de “ir a la derecha”

Solución: Guardar experiencias en un buffer y muestrear aleatoriamente.

```
[83]: class ReplayBuffer:
    """Buffer para almacenar y muestrear experiencias."""

    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """Añade una experiencia al buffer."""
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        """Muestra un batch aleatorio."""
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)
        return (
            np.array(states),
            np.array(actions),
            np.array(rewards),
            np.array(next_states),
            np.array(dones)
        )
```

```

    )

    def __len__(self):
        return len(self.buffer)

# Demo
buffer = ReplayBuffer(capacity=1000)

# Simular algunas experiencias
for i in range(50):
    state = np.random.randn(4)
    action = random.randint(0, 1)
    reward = random.random()
    next_state = np.random.randn(4)
    done = random.random() > 0.9
    buffer.push(state, action, reward, next_state, done)

print(f"Buffer tiene {len(buffer)} experiencias")

# Muestrear batch
states, actions, rewards, next_states, dones = buffer.sample(batch_size=8)
print(f"\nBatch muestreado:")
print(f" States shape: {states.shape}")
print(f" Actions: {actions}")

```

Buffer tiene 50 experiencias

Batch muestreado:

States shape: (8, 4)

Actions: [0 1 1 1 1 1 1 0]

6 PARTE 5: DQN - Tu Agente Jugando (30 min)

Ahora implementaremos **Deep Q-Network (DQN)** completo para entrenar un agente que aprenda a jugar.

6.1 Ejercicio 5.1: Red Q-Network

La red Q toma un **estado** y devuelve el **valor Q** para cada acción posible.

[84]: *# EJERCICIO 5.1: Implementar la Q-Network*

```

class QNetwork(nn.Module):
    """

```

Red neuronal que aproxima $Q(s, a)$.

Input: estado (state_size)

Output: Q-valor para cada acción (n_actions)

Arquitectura:

- Linear(state_size, 64) + ReLU

- Linear(64, 64) + ReLU

- Linear(64, n_actions)

"""

```
def __init__(self, state_size, n_actions):
    super().__init__()
    # TODO: Define las capas
    self.fc1 = nn.Linear(state_size, 64)
    self.fc2 = nn.Linear(64, 64)
    self.fc3 = nn.Linear(64, n_actions)
    self.relu = nn.ReLU()

    def forward(self, x):
        # TODO: Define el forward pass
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)

        return x

# Test
q_net = QNetwork(state_size=4, n_actions=2)
test_state = torch.randn(1, 4)
q_values = q_net(test_state)
print(f"Q-Network creada")
print(f"  Input: {test_state.shape}")
print(f"  Output: {q_values.shape}")
print(f"  Q-values: {q_values.detach().numpy()}")
```

Q-Network creada

Input: torch.Size([1, 4])

Output: torch.Size([1, 2])

Q-values: [[0.08573814 0.09878105]]

[85]: *# VERIFICACIÓN 5.1*

```
assert q_values.shape == (1, 2), f"Output debe ser (1, 2), es {q_values.shape}"
params = sum(p.numel() for p in q_net.parameters())
```

```

assert 4000 < params < 6000, f"Número de parámetros inesperado: {params}"
print(f"Ejercicio 5.1 CORRECTO")
print(f"Parámetros: {params}")

```

Ejercicio 5.1 CORRECTO

Parámetros: 4610

6.2 Ejercicio 5.2: Agente DQN Completo

```

[86]: class DQNAgent:
    """
    Agente DQN con Experience Replay y Target Network.
    """

    def __init__(self, state_size, n_actions):
        self.state_size = state_size
        self.n_actions = n_actions

        # Hiperparámetros
        self.gamma = 0.99          # Factor de descuento
        self.epsilon = 1.0         # Exploración inicial
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.batch_size = 32

        # Redes
        self.q_network = QNetwork(state_size, n_actions).to(DEVICE)
        self.target_network = QNetwork(state_size, n_actions).to(DEVICE)
        self.target_network.load_state_dict(self.q_network.state_dict())
        self.target_network.eval() # Target network en modo evaluación

        # Optimizer
        self.optimizer = optim.Adam(self.q_network.parameters(), lr=self.
↪learning_rate)

        # Replay Buffer
        self.memory = ReplayBuffer(capacity=10000)

    def act(self, state):
        """
        Selecciona acción usando -greedy.

        Con probab : acción aleatoria
        Con probab 1- : mejor acción según Q
        """
        if random.random() < self.epsilon:

```

```

        return random.randint(0, self.n_actions - 1) # Acción aleatoria
    else:
        # Convertir estado a tensor y mover al dispositivo
        state_t = torch.FloatTensor(state).unsqueeze(0).to(DEVICE)
        self.q_network.eval() # Poner en modo evaluación para inferencia
        with torch.no_grad():
            q_values = self.q_network(state_t)
        self.q_network.train() # Volver a modo entrenamiento
        return q_values.argmax(1).item() # Mejor acción según Q

def remember(self, state, action, reward, next_state, done):
    """Guarda experiencia en el buffer."""
    self.memory.push(state, action, reward, next_state, done)

def replay(self):
    """
    Entrena con un batch del replay buffer.

    Target:  $r + \gamma \max_{a'} Q_{\text{target}}(s', a')$ 
    """
    if len(self.memory) < self.batch_size:
        return 0.0

    # Muestrear batch
    states, actions, rewards, next_states, dones = self.memory.sample(self.
↪ batch_size)

    # Convertir a tensores y mover al dispositivo
    states = torch.FloatTensor(states).to(DEVICE)
    actions = torch.LongTensor(actions).to(DEVICE)
    rewards = torch.FloatTensor(rewards).to(DEVICE)
    next_states = torch.FloatTensor(next_states).to(DEVICE)
    dones = torch.FloatTensor(dones).to(DEVICE)

    # Calcular Q-valores actuales
    current_q_values = self.q_network(states) # Todos los Q-valores
    current_q = current_q_values.gather(1, actions.unsqueeze(1)).squeeze() ↪
↪ # Solo la acción tomada

    # Calcular Q-valores objetivo (target)
    with torch.no_grad():
        next_q = self.target_network(next_states).max(1)[0]
        target_q = rewards + (1 - dones) * self.gamma * next_q

    # Calcular loss y actualizar
    loss = F.mse_loss(current_q, target_q)
    self.optimizer.zero_grad()

```

```

        loss.backward()
        self.optimizer.step()

    return loss.item()

def update_target(self):
    """Copia pesos de q_network a target_network."""
    self.target_network.load_state_dict(self.q_network.state_dict())

def decay_epsilon(self):
    """Reduce exploración gradualmente."""
    self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

```

6.3 Ejercicio 5.3: Entrenar el Agente

```

[87]: # EJERCICIO 5.3: Ciclo de entrenamiento DQN

def train_dqn(env, agent, episodes=200):
    """Entrena un agente DQN."""
    rewards_history = []

    for episode in range(episodes):
        state = env.reset()
        total_reward = 0
        done = False

        while not done:
            # TODO: 1. Seleccionar acción
            action = agent.act(state) # TODO: agent.act(state)

            # TODO: 2. Ejecutar acción en el entorno
            next_state, reward, done = env.step(action) # TODO: env.
            ↪ step(action)

            # TODO: 3. Guardar experiencia
            # TODO: agent.remember(state, action, reward, next_state, done)
            agent.remember(state, action, reward, next_state, done)

            # TODO: 4. Entrenar con replay
            # TODO: agent.replay()
            agent.replay()

            state = next_state
            total_reward += reward

        rewards_history.append(total_reward)
        agent.decay_epsilon()

```

```

    # Actualizar target network cada 10 episodios
    if episode % 10 == 0:
        agent.update_target()

    if (episode + 1) % 50 == 0:
        avg = np.mean(rewards_history[-50:])
        print(f"Ep {episode+1:3d} | Avg Reward: {avg:.2f} | : {agent.
↪epsilon:.3f}")

    return rewards_history

# Entrenar
print("="*60)
print("ENTRENAMIENTO DQN")
print("="*60 + "\n")

env = SimpleGridWorld()
agent = DQNAgent(state_size=4, n_actions=2)

rewards = train_dqn(env, agent, episodes=200)

```

```

=====
ENTRENAMIENTO DQN
=====

```

```

Ep  50 | Avg Reward: 6.55 | : 0.778
Ep 100 | Avg Reward: 6.36 | : 0.606
Ep 150 | Avg Reward: 8.70 | : 0.471
Ep 200 | Avg Reward: 9.44 | : 0.367

```

```

[88]: # Visualizar aprendizaje
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(rewards, alpha=0.6, label='Reward por episodio')
# Media móvil
window = 20
if len(rewards) > window:
    avg = np.convolve(rewards, np.ones(window)/window, mode='valid')
    plt.plot(range(window-1, len(rewards)), avg, 'r-', linewidth=2,
↪label=f'Media móvil ({window})')
plt.xlabel('Episodio')
plt.ylabel('Reward Total')
plt.title('Curva de Aprendizaje DQN')
plt.legend()

```



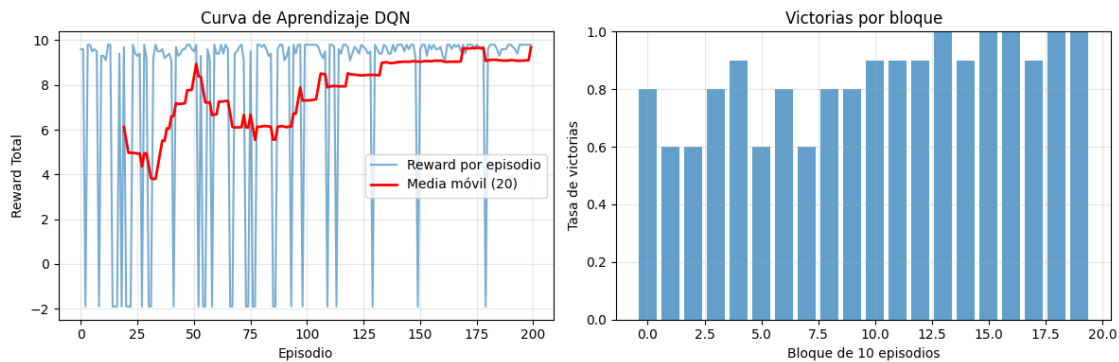
```

plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
# Calcular tasa de victorias por ventana
wins = [1 if r > 0 else 0 for r in rewards]
win_rate = []
for i in range(0, len(wins), 10):
    win_rate.append(sum(wins[i:i+10]) / min(10, len(wins)-i))
plt.bar(range(len(win_rate)), win_rate, alpha=0.7)
plt.xlabel('Bloque de 10 episodios')
plt.ylabel('Tasa de victorias')
plt.title('Victorias por bloque')
plt.ylim(0, 1)
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



6.4 Ejercicio 5.4: Evaluar el Agente Entrenado

```

[89]: # EJERCICIO 5.4: Evaluar el agente (sin exploración)

def evaluar_agente(env, agent, n_episodios=20, render=True):
    """Evalúa el agente sin exploración (=0)."""
    epsilon_backup = agent.epsilon
    agent.epsilon = 0 # Solo explotación

    victorias = 0

    for ep in range(n_episodios):
        state = env.reset()
        done = False
        total_reward = 0

```

```

    if render and ep < 3: # Solo mostrar primeros 3
        print(f"\nEpisodio {ep+1}:")
        env.render()

    while not done:
        action = agent.act(state)
        state, reward, done = env.step(action)
        total_reward += reward

        if render and ep < 3:
            env.render()

    if total_reward > 0:
        victorias += 1

    agent.epsilon = epsilon_backup
    return victorias / n_episodios

# Evaluar
print("="*60)
print("EVALUACIÓN DEL AGENTE ENTRENADO")
print("="*60)

win_rate = evaluar_agente(env, agent, n_episodios=20, render=True)
print(f"\n" + "="*60)
print(f"RESULTADO: {win_rate*100:.0f}% de victorias")
print("="*60)

```

```

=====
EVALUACIÓN DEL AGENTE ENTRENADO
=====

```

Episodio 1:

```

[A _ _ G] Pos: 0
[_ A _ G] Pos: 1
[_ _ A G] Pos: 2
[_ _ _ *] Pos: 3

```

Episodio 2:

```

[A _ _ G] Pos: 0
[_ A _ G] Pos: 1
[_ _ A G] Pos: 2
[_ _ _ *] Pos: 3

```

Episodio 3:

```

[A _ _ G] Pos: 0

```

```
[_ A _ G] Pos: 1
[_ _ A G] Pos: 2
[_ _ _ *] Pos: 3
```

```
=====
RESULTADO: 100% de victorias
=====
```

7 Resumen y Siguietes Pasos

7.1 Lo que has aprendido:

Parte	Conceptos
1	Tensores: creación, operaciones, reshape, dispositivos
2	Autograd, nn.Module, forward/backward pass
3	Entrenamiento supervisado completo (Iris)
4	Diferencias RL vs supervisado, -greedy, replay buffer
5	DQN completo: Q-Network, agente, entrenamiento

7.2 Para ir más allá:

1. **Juegos más complejos:** Intenta con entornos de Gymnasium (CartPole, LunarLander)
2. **Mejoras de DQN:** Double DQN, Dueling DQN, Prioritized Experience Replay
3. **Policy Gradient:** Otros algoritmos como REINFORCE, A2C, PPO
4. **Juegos visuales:** Usar CNNs para aprender directamente de píxeles

Consulta los notebooks de teoría para más detalles: - DL_02_pytorch_basico.ipynb - Referencia completa de PyTorch - DL_05_redes_para_RL.ipynb - DQN y arquitecturas para RL